

cognizant

2027 Nurture Partner Network (NPN) MuleSoft

Hackathon Challenge – Ecommerce Order Fulfilment Pipeline

© 2026-2027 Cognizant | Private

Hackathon Guidelines

1. Each team shall complete and submit the hackathon project within five calendar days, commencing on the date the hackathon requirements are communicated.
2. All developed APIs and apps must be in a runnable state, with MUnit tests included.
3. Every team member must have the runnable code on their own laptop.
4. Each member can have their own way of implementation as well.
5. Running the API on your Anypoint trial account runtime is highly preferred. If you face any issue with your Anypoint account, run the app on a local setup instead — note that a local run will be scored lower than a CloudHub runtime deployment.

Technical Standards and Best Practices

1. Prepare High Level Design document.
2. API-LED Architecture.
3. Global error handling.
4. Use RAML wherever applicable and API Kit router.
5. MUnit covering 100% of components. If not possible, need valid reasons.
6. Auto Doiscovery.
7. Externalizing properties, secure properties and dwl files.
8. Appropriate Logging levels.
9. Apply applicable API policies for performance & security.
10. Use of appropriate private and sub flows.
11. Use core components based on the need like dwl, for-each/parallel for-each, etc.,
12. Domain app for studio run applications.
13. Basic alerts, logging & monitoring.
14. Code should be maintained in Git. And everyone should be able to commit, pull, raise merge requests, merge conflicts, etc.,
15. Use batch processing, caching, object store wherever required in your project.
16. Standard names for components in the flow.

Scenario: E-Commerce — Order Fulfillment Pipeline

Accept e-commerce orders through an API, persist them, fulfil them asynchronously off a queue, generate an invoice, and email the customer a confirmation.

The design challenge is the boundary between the synchronous and the asynchronous halves of the pipeline. The storefront needs an answer in a few hundred milliseconds, so the API flow may only validate the order, reserve nothing more than a row in the database and hand the work to a queue. Everything expensive — stock reservation, invoice generation, file writing and email delivery — belongs to the consumer. The order must survive a restart at

© 2026-2027 Cognizant | Private

any point: once the API has returned 202 the customer believes the order exists, so the database write and the queue publish have to be tied together rather than performed hopefully in sequence. The invoice file is a durable business artefact, so it must be written exactly once even when a message is redelivered.

Components to Be Used

1. **HTTP Listener** — exposes POST /api/v1/orders for order capture and GET /api/v1/orders/{orderId} for status polling by the storefront.
2. **Database Connector** — persists the order header, order lines and invoice metadata, and holds the fulfilment status that drives the flow.
3. **JMS / ActiveMQ Connector** — carries accepted orders to the fulfilment consumer on order.fulfilment.queue, and completed fulfilments to the notification consumer on order.notify.queue.
4. **File Connector** — writes the generated invoice to the invoice output directory and archives a copy under a dated folder structure.
5. **Email Connector (SMTP)** — sends the order confirmation to the customer with the generated invoice attached.

End-to-End Integration Flow

```
SYNCHRONOUS (storefront facing, target under 400 ms)

  HTTP POST /api/v1/orders
    -> schema and business validation
    -> DATABASE  insert orders + order_items (status ACCEPTED)
    -> JMS       publish to order.fulfilment.queue
    -> HTTP 202 Accepted { orderId, status, statusUrl }

ASYNCHRONOUS STAGE 1 - fulfilment consumer

  JMS consume order.fulfilment.queue
    -> DATABASE  reserve stock, set status RESERVED
    -> generate invoice document
    -> FILE      write invoice to /data/invoices/out
    -> FILE      archive copy to /data/invoices/archive/yyyy/MM/dd
    -> DATABASE  insert invoices row, set status FULFILLED
    -> JMS       publish to order.notify.queue

ASYNCHRONOUS STAGE 2 - notification consumer

  JMS consume order.notify.queue
    -> DATABASE  read order, customer and invoice path
    -> FILE      read invoice file as attachment
    -> EMAIL     send confirmation to customer
    -> DATABASE  set notified_at

FAILURE PATHS

  validation failure        -> HTTP 400, nothing persisted
  stock unavailable         -> status BACKORDERED, backorder email
```

© 2026-2027 Cognizant | Private

```
retries exhausted        -> order.fulfilment.queue.DLQ + ops alert
email permanently failed  -> status FULFILLED_NOT_NOTIFIED, ops alert
```

Functional Requirements

1. **Order intake API -** Expose POST /api/v1/orders described by a RAML specification and implemented with APIkit so that the request contract is enforced by the router rather than by hand-written checks.
2. **Request validation -** Reject an order that has no line items, a quantity that is zero or negative, a currency that is not on the supported list, or a line total that does not equal quantity multiplied by unit price.
3. **Idempotent capture -** Treat the client-supplied X-Request-Id header as an idempotency key. A repeated header must return the original orderId and the original status without creating a second order.
4. **Atomic accept -** Perform the database insert and the JMS publish inside a single XA transaction or implement an outbox table polled by a separate publisher flow. An order must never exist in the database without a corresponding queue message, and a queue message must never reference an order that was never committed.
5. **Fast response -** Return 202 Accepted with the orderId, the current status and a status URL. Do not wait for stock reservation, invoice generation or email delivery before responding.
6. **Status endpoint -** Expose GET /api/v1/orders/{orderId} returning the current status, the status timestamps and the invoice number once one exists. Return 404 for an unknown orderId.
7. **Stock reservation -** In the fulfilment consumer, decrement available stock for every line with a conditional update that cannot drive the quantity below zero, and treat the affected row count as the authority on whether the reservation succeeded.
8. **Partial availability -** When at least one line cannot be reserved, do not fail the whole order. Reserve what is available, mark the remaining lines BACKORDERED and record the shortfall against each line.
9. **Invoice generation -** Build an invoice containing the invoice number, the invoice date, the billing address, every line with its tax treatment, the tax subtotals by rate and the payable total. The invoice number must come from a database sequence so that numbering has no gaps.
10. **Invoice files write -** Write the invoice with a temporary name and rename it into place on completion, so that a downstream reader never sees a half-written file. Archive a second copy under a dated directory and never overwrite an existing archived invoice.
11. **Exactly-once side effects -** Record the JMS message identifier in a processed_messages table before performing the file write, and skip the whole unit of work when the identifier is already present. A redelivered message must not produce a second invoice file or a second invoice number.
12. **Notification decoupling -** Publish to order.notify.queue rather than sending mail from the fulfilment consumer, so that a slow or unavailable SMTP server cannot hold a database transaction open or cause stock to be reserved twice.
13. **Confirmation email -** Send an email containing the order number, the order date, the delivery address, the itemised lines, the payable total and the expected dispatch window, with the invoice file attached. Record the send timestamp against the order.

© 2026-2027 Cognizant | Private

14. **Retry policy -** Apply a bounded retry with exponential backoff to the database, file and SMTP operations. Distinguish a transient failure, which is retried, from a permanent failure such as a rejected recipient address, which is not.
15. **Dead letter handling -** Route a message that exhausts its retries to order.fulfilment.queue.DLQ together with the failure reason, the attempt count and the original payload and raise an operational alert.
16. **Correlation -** Generate a correlation identifier at the HTTP boundary and propagate it through the queue message, every log line, the invoice metadata and the email headers so that one order can be traced end to end.
17. **Secure configuration -** Hold the database credentials, the ActiveMQ credentials and the SMTP credentials in an encrypted properties file and select the environment at deployment time. No credential may appear in the flow XML.
18. **Logging discipline -** Log the order identifier, the correlation identifier, the stage and the outcome at each transition. Never log the full card or payment details present on the inbound payload.
19. **Test coverage -** Provide MUnit tests for the happy path, the validation failures, the duplicate request, the partial availability case, the redelivered message case and the dead letter path, with the database, JMS and SMTP endpoints mocked.

Sample Order Request

```
   POST /api/v1/orders HTTP/1.1
Content-Type: application/json
X-Request-Id: 8f14e45f-ea0c-4c11-9d7a-2b6c41f0a913

{
 "customer": {
  "customerId": "CUST-77219",
  "name": "Meena Raghavan",
  "email": "meena.raghavan@example.com",
  "phone": "+91-98400-11223"
 },
 "billingAddress": {
  "line1": "14 Kasturba Road",
  "city": "Chennai",
  "state": "TN",
  "postalCode": "600018",
  "country": "IN",
  "gstin": "33AAACR5055K1ZD"
 },
 "shippingAddress": {
  "line1": "14 Kasturba Road",
  "city": "Chennai",
  "state": "TN",
  "postalCode": "600018",
  "country": "IN"
 },
 "currency": "INR",
 "items": [
  {
   "sku": "SKU-ELEC-1188",
   "productName": "Wireless Keyboard K380",
   "quantity": 2,
   "unitPrice": 2499.00,
```

© 2026-2027 Cognizant | Private

```
      "taxRatePercent": 18.0,
      "lineTotal": 4998.00
    },
    {
      "sku": "SKU-ELEC-2041",
      "productName": "USB-C Hub 7-in-1",
      "quantity": 1,
      "unitPrice": 3299.00,
      "taxRatePercent": 18.0,
      "lineTotal": 3299.00
    }
  ],
  "orderTotal": 8297.00,
  "paymentMethod": "PREPAID_UPI",
  "paymentReference": "UPI-2026-0911-778213"
}
```

Order Accepted Response

```
HTTP/1.1 202 Accepted
Content-Type: application/json
X-Correlation-Id: 8f14e45f-ea0c-4c11-9d7a-2b6c41f0a913

{
  "orderId": "ORD-2026-0009142",
  "status": "ACCEPTED",
  "acceptedAt": "2026-09-11T10:24:18.402+05:30",
  "statusUrl": "/api/v1/orders/ORD-2026-0009142",
  "message": "Order accepted and queued for fulfilment."
}

# Status polled a few seconds later
HTTP/1.1 200 OK

{
  "orderId": "ORD-2026-0009142",
  "status": "FULFILLED",
  "invoiceNumber": "INV-2026-0004471",
  "acceptedAt": "2026-09-11T10:24:18.402+05:30",
  "fulfilledAt": "2026-09-11T10:24:23.771+05:30",
  "notifiedAt": "2026-09-11T10:24:25.118+05:30",
  "lines": [
    { "sku": "SKU-ELEC-1188", "quantity": 2, "status": "RESERVED" },
    { "sku": "SKU-ELEC-2041", "quantity": 1, "status": "RESERVED" }
  ]
}
```

Fulfilment Queue Message

```
# Destination : order.fulfilment.queue
# JMSType     : OrderAccepted
# Properties  : orderId, customerId, priority, correlationId

{
  "eventType": "ORDER_ACCEPTED",
  "eventId": "evt-5c1f8a2e-7d90-4b13-8a44-0e2f91c7b640",
  "correlationId": "8f14e45f-ea0c-4c11-9d7a-2b6c41f0a913",
```

© 2026-2027 Cognizant | Private

```
  "occurredAt": "2026-09-11T10:24:18.402+05:30",
  "orderId": "ORD-2026-0009142",
  "customerId": "CUST-77219",
  "customerEmail": "meena.raghavan@example.com",
  "currency": "INR",
  "orderTotal": 8297.00,
  "items": [
    { "sku": "SKU-ELEC-1188", "quantity": 2, "unitPrice": 2499.00 },
    { "sku": "SKU-ELEC-2041", "quantity": 1, "unitPrice": 3299.00 }
  ],
  "attempt": 1
}

# Notification message published after fulfilment
# Destination : order.notify.queue

{
  "eventType": "ORDER_FULFILLED",
  "correlationId": "8f14e45f-ea0c-4c11-9d7a-2b6c41f0a913",
  "orderId": "ORD-2026-0009142",
  "invoiceNumber": "INV-2026-0004471",
  "invoicePath": "/data/invoices/out/INV-2026-0004471.txt",
  "backorderedLines": 0
}
```

Database Table Structure

```
CREATE TABLE orders (
  order_id         VARCHAR(24)  PRIMARY KEY,
  customer_id      VARCHAR(20)  NOT NULL,
  customer_email   VARCHAR(150) NOT NULL,
  currency         CHAR(3)      NOT NULL,
  order_total      DECIMAL(12,2) NOT NULL,
  payment_method   VARCHAR(30),
  payment_ref      VARCHAR(60),
  status           VARCHAR(24)  NOT NULL,
  invoice_number   VARCHAR(24),
  request_id       VARCHAR(64)  NOT NULL,
  correlation_id   VARCHAR(64)  NOT NULL,
  accepted_at      TIMESTAMP    NOT NULL,
  fulfilled_at     TIMESTAMP,
  notified_at      TIMESTAMP,
  CONSTRAINT uq_orders_request UNIQUE (request_id),
  CONSTRAINT ck_orders_status CHECK (status IN
    ('ACCEPTED','RESERVED','FULFILLED','BACKORDERED',
     'FULFILLED_NOT_NOTIFIED','FAILED'))
);

CREATE TABLE order_items (
  order_item_id    BIGINT       PRIMARY KEY AUTO_INCREMENT,
  order_id         VARCHAR(24)  NOT NULL,
  sku              VARCHAR(30)  NOT NULL,
  product_name     VARCHAR(150) NOT NULL,
  quantity         INT          NOT NULL,
  unit_price       DECIMAL(12,2) NOT NULL,
  tax_rate_percent DECIMAL(5,2) NOT NULL,
  line_total       DECIMAL(12,2) NOT NULL,
  reserved_qty     INT          NOT NULL DEFAULT 0,
```

© 2026-2027 Cognizant | Private

```
  line_status      VARCHAR(20)  NOT NULL DEFAULT 'PENDING',
  CONSTRAINT fk_items_order FOREIGN KEY (order_id)
    REFERENCES orders(order_id),
  CONSTRAINT ck_items_qty CHECK (quantity > 0)
);

CREATE TABLE inventory_stock (
  sku              VARCHAR(30)  PRIMARY KEY,
  available_qty    INT          NOT NULL,
  reserved_qty     INT          NOT NULL DEFAULT 0,
  reorder_point    INT          NOT NULL DEFAULT 0,
  updated_at       TIMESTAMP    NOT NULL,
  CONSTRAINT ck_stock_nonneg CHECK (available_qty >= 0)
);

CREATE TABLE invoices (
  invoice_number   VARCHAR(24)  PRIMARY KEY,
  order_id         VARCHAR(24)  NOT NULL,
  invoice_date     DATE         NOT NULL,
  taxable_amount   DECIMAL(12,2) NOT NULL,
  tax_amount       DECIMAL(12,2) NOT NULL,
  payable_total    DECIMAL(12,2) NOT NULL,
  file_path        VARCHAR(300) NOT NULL,
  archive_path     VARCHAR(300) NOT NULL,
  generated_at     TIMESTAMP    NOT NULL,
  CONSTRAINT uq_invoice_order UNIQUE (order_id),
  CONSTRAINT fk_invoice_order FOREIGN KEY (order_id)
    REFERENCES orders(order_id)
);

CREATE TABLE processed_messages (
  jms_message_id   VARCHAR(120) PRIMARY KEY,
  consumer_name    VARCHAR(60)  NOT NULL,
  order_id         VARCHAR(24),
  processed_at     TIMESTAMP    NOT NULL
);

-- Conditional reservation: affected row count is the authority
UPDATE inventory_stock
   SET available_qty = available_qty - :qty,
       reserved_qty  = reserved_qty  + :qty,
       updated_at    = CURRENT_TIMESTAMP
 WHERE sku = :sku
   AND available_qty >= :qty;
```

Generated Invoice File Layout

```
# File : /data/invoices/out/INV-2026-0004471.txt
# Written as INV-2026-0004471.txt.tmp then renamed into place.

============================================================
                    TAX INVOICE
============================================================
Invoice Number : INV-2026-0004471
Invoice Date   : 11-Sep-2026
Order Number   : ORD-2026-0009142
Order Date     : 11-Sep-2026
```

© 2026-2027 Cognizant | Private

```
SELLER
  NPN Retail Private Limited
  Plot 22, Ambattur Industrial Estate, Chennai 600058
  GSTIN : 33AABCN1234M1Z7

BILL TO
  Meena Raghavan
  14 Kasturba Road, Chennai, TN 600018, IN
  GSTIN : 33AAACR5055K1ZD

-------------------------------------------------------------
LINE  SKU            QTY   RATE      TAX%     AMOUNT
-------------------------------------------------------------
1     SKU-ELEC-1188      2   2499.00   18.0    4998.00
      Wireless Keyboard K380
2     SKU-ELEC-2041      1   3299.00   18.0    3299.00
      USB-C Hub 7-in-1
-------------------------------------------------------------
                    Taxable Amount   :    7031.36
                    CGST 9%          :     632.82
                    SGST 9%          :     632.82
                    ------------------------------
                    PAYABLE TOTAL    :    8297.00
=============================================================
Payment : PREPAID_UPI  Ref UPI-2026-0911-778213
Status  : PAID
=============================================================

# Archive copy, never overwritten:
#   /data/invoices/archive/2026/09/11/INV-2026-0004471.txt
```

Order Confirmation Email Format

```
**To:** {{customerEmail}}
**From:** orders@npnretail.example.com
**Subject:** Order {{orderId}} confirmed – dispatch by {{dispatchBy}}
**Attachment:** {{invoiceNumber}}.txt

Dear {{customerName}},

Thank you for your order. We have received your payment and your
order is now being prepared for dispatch.

Order Number   : {{orderId}}
Order Date     : {{orderDate}}
Invoice Number : {{invoiceNumber}}
Payable Total  : {{currency}} {{payableTotal}}

Items
{{#each items}}
  {{quantity}} x {{productName}} ({{sku}})
      {{currency}} {{lineTotal}}   [{{lineStatus}}]
{{/each}}

Delivery Address
  {{shippingLine1}}
  {{shippingCity}}, {{shippingState}} {{shippingPostalCode}}
```

© 2026-2027 Cognizant | Private

```
Expected dispatch : {{dispatchBy}}
Track your order  : {{statusUrl}}

Your tax invoice is attached to this email.

Regards,
NPN Retail Customer Care

# Backorder variant subject line:
#   Order {{orderId}} confirmed – {{backorderedCount}} item(s) to
#   follow
```

Basic Negative Scenarios to Be Covered

1. **Empty order -** A request with an empty items array is rejected with 400 and nothing is written to the database or the queue.
2. **Invalid quantity or price -** A zero, negative or non-numeric quantity, or a negative unit price, is rejected with a field-level error that names the offending line index.
3. **Line total mismatch -** A line whose lineTotal does not equal quantity multiplied by unitPrice is rejected, and an orderTotal that does not equal the sum of the line totals is rejected.
4. **Unknown SKU -** An order referencing a SKU that does not exist in inventory_stock is rejected with 422 and the unknown SKU is named.
5. **Duplicate request -**The same X-Request-Id submitted twice returns the original orderId and status, and exactly one row exists in orders.
6. **Malformed JSON -** A body that is not valid JSON returns 400 with a parse error and is not retried.
7. **Unsupported currency -** A currency outside the configured list is rejected rather than defaulted.
8. **Database unavailable at intake -** When the database cannot be reached, the API returns 503 with a Retry-After header and publishes nothing to the queue.
9. **Broker unavailable at intake -** When ActiveMQ cannot be reached, the transaction rolls back or the outbox retains the record, so the order is never left in ACCEPTED with no queue message.
10. **Insufficient stock -** A line that cannot be reserved is marked BACKORDERED, the reservable lines still reserve, available_qty never goes negative, and the customer receives the backorder variant of the email.
11. **Concurrent reservation -** Two orders for the last unit of the same SKU are processed simultaneously; exactly one reserves it and the other is backordered.
12. **Redelivered message -** A fulfilment message delivered twice produces one invoice number, one invoice file, one archive copy and one email.
13. **Invoice directory not writable -** A file write failure is retried, and on exhaustion the order is left in RESERVED and routed to the dead letter queue rather than being marked FULFILLED.

© 2026-2027 Cognizant | Private

14. **Partially written invoice -** A consumer killed midway through the file write leaves only a .tmp file, which is ignored by readers and cleaned up on the next run.
15. **Archive collision -** An invoice number that already exists in the archive directory raises an alert instead of silently overwriting the earlier file.
16. **SMTP unavailable -** Mail failure is retried with backoff; on exhaustion the order is set to FULFILLED_NOT_NOTIFIED and an operational alert is raised, but stock stays reserved and the invoice is not regenerated.
17. **Rejected recipient -** A hard SMTP rejection for an invalid mailbox is treated as permanent, is not retried, and is recorded against the order.
18. **Poison message -** A queue message whose body cannot be parsed goes straight to the dead letter queue without consuming the full retry budget.
19. **Unknown order status query -** GET on an orderId that does not exist returns 404 with a clear message rather than an empty 200.
20. **Additional negative scenarios -** relevant to the implementation may be covered and should be documented in the submission.

Acceptance Criteria

1. POST /api/v1/orders returns 202 with an orderId in under 400 milliseconds at the demonstration load, measured without the consumer running.
2. Every accepted order has exactly one row in orders, a matching set of order_items rows and exactly one message on order.fulfilment.queue.
3. Stopping the consumer, submitting ten orders and then starting the consumer results in all ten being fulfilled with no loss and no duplication.
4. Invoice numbers issued across the demonstration form an unbroken sequence with no gaps and no repeats.
5. One invoice file exists in the output directory and one in the dated archive directory for every fulfilled order, and no .tmp file remains.
6. Replaying the same fulfilment message ten times still yields one invoice file, one invoices row and one email.
7. available_qty in inventory_stock is never negative at any point during a concurrent load test.
8. A backordered order still produces an invoice for the reserved lines and an email that names the items to follow.
9. Killing the consumer mid-fulfilment and restarting it completes the order exactly once.
10. A message that fails repeatedly appears on order.fulfilment.queue.DLQ with the failure reason and attempt count, and the order is not falsely marked FULFILLED.
11. With SMTP stopped, orders still reach FULFILLED and the invoice files are still produced; the status becomes FULFILLED_NOT_NOTIFIED.
12. The correlation identifier from the request appears in the queue message, the log lines, the invoices row and the email headers.

© 2026-2027 Cognizant | Private

13. No credential appears in the deployable artefact in plain text, and the same artefact deploys to two environments by property change alone.
14. MUnit tests cover the happy path, validation failures, duplicate requests, insufficient stock, redelivery and the dead letter path, and they pass in a single run.

© 2026-2027 Cognizant | Private